Art Abstrait et Arbres
Projet d'ASD3

A réaliser en binôme

Octobre 2026

Consignes importantes :

- Ce projet doit être réalisé en Java 1.8, en binôme, et doit être obligatoirement écrit en intégralité par le binôme qui le présente[1]. L'appel à des structures de données optimisées existantes (du genre `TreeSet` ou `HashMap`, `HashSet` etc.) est interdit : vous devez implémenter vous-mêmes vos structures de données autres que tableaux et listes.
- La complexité de vos algorithmes/programmes est très importante. Seules les solutions assurant une meilleure complexité (calculée comme d'habitude, dans le pire des cas) seront acceptées.

1 Introduction

Piet Mondrian est un peintre du début du 20ème siècle, pionnier de l'art abstrait. Son oeuvre est caractérisée par des oeuvres très géométriques, comme celle présentée en Figure 1[2].

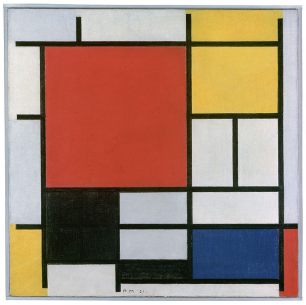
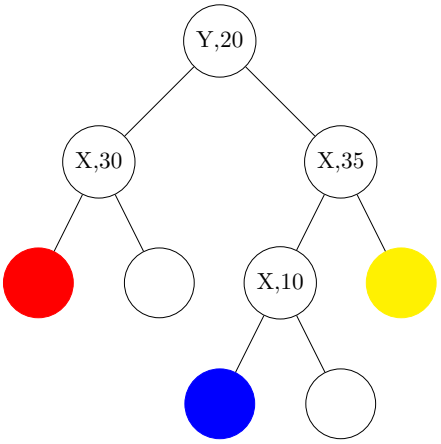


Figure 1: Composition avec grand plan rouge, jaune, noir, gris et bleu

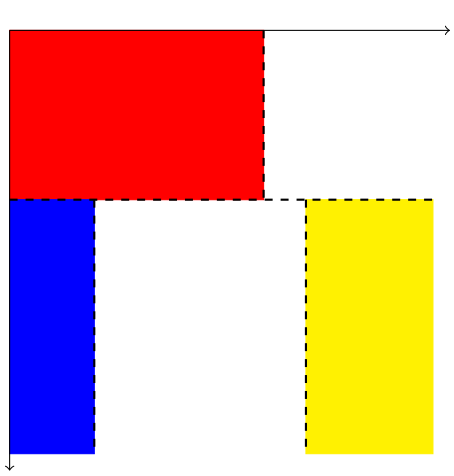
Le but de ce projet est d'utiliser les structures vues en cours pour générer des images à la manière des tableaux de Mondrian. À la fin du projet vous proposerez un outil pour générer des peintures de manière aléatoire, avec plusieurs méthodes de génération, une méthode pour convertir un tableau en représentation textuelle et vice-versa, et un rapport qui montre ce que vous arrivez à générer avec votre travail.

[1] L'utilisation des intelligences artificielles est interdite. Toute ressemblance - même mineure - entre deux programmes, ou entre un programme et du code sur Internet, ou entre un programme et du code produit par une IA, sera considérée comme non-fortuite et entraînera automatiquement la note de 0 sur la partie concernée ainsi que sur toute autre partie utilisant, même très peu, la partie concernée. Une pénalité supplémentaire de 5 points sera également appliquée.

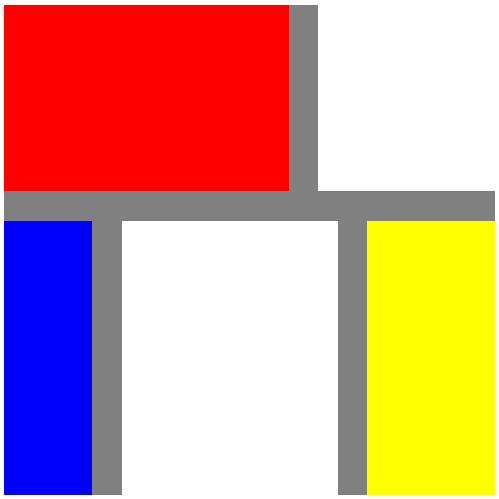
[2] `https://fr.wikipedia.org/wiki/Composition_avec_grand_plan_rouge,_jaune,_noir,_gris_et_bleu`



(a) Arbre



(b) Représentation



(c) Tableau généré

2 Projet

2.1 Conversion arbre→peinture

Les arbres que vous allez implémenter et utiliser dans ce projet sont une variante des 2d-arbres vus en cours. Dans votre cours les 2d-arbres alternaient une division sur les X, et une division sur les Y, et contenaient des points pour connaître les coordonnées de la division. Ici, l'arbre doit permettre de :

- faire une découpe selon l'axe des Y au noeud racine;
- faire plusieurs découpes selon le même axe d'affilée;
- représenter des couleurs dans les noeuds.

Un exemple d'arbre à représenter est donné en Figure 2a. Cet arbre a une représentation, à la manière de celles que vous avez vues en cours, donnée en Figure 2b. À partir de cet arbre un tableau peut être généré. Des lignes doivent être ajoutées, avec une largeur `largeurLigne` (en nombre de pixels), pour finalement obtenir une image, comme celle en Figure 2c.

Important. Le repère orthogonal est obligatoirement, pour ce projet, celui de la Figure 2b (origine (0,0) en haut à gauche, axe des X vers la droite, axe des Y vers le bas). Vous devez utiliser ce repère, et fournir les images demandées selon ce repère.

2.2 Génération d'arbres

Pour générer un arbre (donc un tableau), on se donne une limite `nbFeuillesAdiviser`, à ne pas dépasser, sur le nombre de feuilles à diviser. On part d'un arbre initial qui contient uniquement une racine/feuille blanche (donc un tableau blanc), et on le transforme en plusieurs étapes. À chaque étape, une feuille de l'arbre courant est sélectionnée pour être divisée en deux, jusqu'à ce que le nombre maximum `nbFeuillesAdiviser` de feuilles divisées soit atteint, ou qu'on ne puisse plus diviser (voir la Section 2.2.1). Lors de la création d'une feuille, on lui assigne une couleur selon des règles précises (voir la Section 2.2.3). Le tableau représenté par l'arbre est ensuite dessiné. Les détails sont fournis ci-dessous.

2.2.1 Choix de la feuille à diviser

On associe à chaque feuille un poids défini comme suit. Soit F une feuille de l'arbre, notons w et h la largeur (calculée le long de l'axe X) et la hauteur (calculée selon l'axe Y) de la région représentée par la feuille. Le poids de la feuille est défini comme

$$poids(F) = \frac{wh}{(w+h)^{1.5}}$$

La feuille choisie pour être divisée est la feuille de plus grand poids parmi les feuilles dont les dimensions sont encore assez grandes pour être divisées. (S'il y a plusieurs feuilles qui satisfont cette condition, alors on choisit de diviser celle dont le coin supérieur gauche a les coordonnées (x, y) les plus petites par ordre lexicographique[3].) L'idée de ce poids est d'essayer de diviser prioritairement les feuilles carrées. En revanche il ne faut pas diviser les régions trop petites, au risque de se retrouver avec uniquement des traits. Pour s'en empêcher, on se donne une valeur `minDimensionCoupe`, et – si la taille dans au moins une des deux dimensions de la région est plus petite que `minDimensionCoupe` – on ne peut pas diviser la feuille. Si, à la fin, toutes les régions sont trop petites et que le nombre de feuilles à diviser `nbFeuillesAdiviser` n'est pas atteint, ce n'est pas grave, la procédure s'arrête sans avoir atteint le nombre de feuilles.

2.2.2 Choix de l'axe/coordonnée de la division

Quand une feuille F est choisie pour être divisée, il faut choisir l'axe de la division, ainsi que la coordonnée. L'axe est choisi en fonction de la forme de la région à diviser. Soient w et h la largeur et la hauteur de la feuille choisie.

Il faut prévoir deux variantes:

- **basique (B):** L'axe choisi est l'axe des X si $\frac{w}{h} > 1$ et l'axe des Y dans le cas contraire.
- **avancée (A):** L'axe choisi est l'axe des X si $\frac{w}{h} > 2$ ou si $\frac{h}{w} \leq 2$, et l'axe des Y si $\frac{h}{w} > 2$ ou si $\frac{w}{h} \leq 2$.

Pour la coordonnée, supposons que F est la r-ème feuille qui est divisée. Soient (x_r, y_r) et (x'_r, y'_r) les coordonnées des points supérieur gauche et inférieur droit de la région représentée par la feuille (de sorte que $w = x'_r - x_r$ et $h = y'_r - y_r$). La coordonnée de découpe est calculée selon les informations lues dans un fichier (dont le nom est fourni en entrée par l'utilisateur), dont les lignes sont de la forme:

$1\ p_1\ q_1\ g_1\ d_1$
$2\ p_2\ q_2\ g_2\ d_2$
...
$r\ p_r\ q_r\ g_r\ d_r$
...
$k\ p_k\ q_k\ g_k\ d_k$

[3]Rappel. L'ordre lexicographique décide que $(a, b) < (a', b')$ si et seulement si soit $a < a'$, soit $a = a'$ et $b < b'$.

où k =`nbFeuillesAdiviser`, et $0.2 \leq p_i, q_i \leq 0.8$ pour tout i. (Les deux autres informations sur chaque ligne sont des couleurs.) La ligne r du fichier indique p_r et q_r. Si l'axe X a été choisi pour F, alors la division se fera selon un axe en position $x_r + \lfloor w * p_r \rfloor$; si l'axe Y a été choisi, alors la division se fera selon un axe en position $y_r + \lfloor h * q_r \rfloor$. Il est sous-entendu que les paramètres `largeurLigne`, `minDimensioncoupe`, p_i, q_i etc. fournis doivent être compatibles les uns avec les autres pour qu'un tableau puisse être obtenu.

2.2.3 Choix des couleurs

Lorsque la r-ème feuille F est divisée, la couleur de ses feuilles gauche et droite respectivement est donnée par les paramètres g_r et d_r respectivement du fichier fourni en entrée.(Rappelez-vous bien que les feuilles gauche et droite sont bien définies, vous ne les choisissez pas au hasard : si la division utilise un axe vertical, le fils gauche correspond à la région de gauche et celui de droite à la région de droite; si la division utilise un axe horizontal, le fils gauche correspond à la région du haut et le fils droit à la région du bas). Ces paramètres sont des lettres qui représentent 6 couleurs : n pour noir, w pour blanc, r pour rouge, j pour jaune, b pour bleu et m pour la même couleur que la feuille qui vient d'être divisée.

2.2.4 Amélioration de la stratégie

Pour le projet, vous devrez implémenter dans un premier temps la méthode définie ci-dessus. Cette méthode a plusieurs défauts (par exemple celui qu'un fichier doit être utilisé pour guider les divisions et les couleurs, ou celui que certaines divisions ne sont pas possibles). Vous devez proposer une nouvelle méthode qui s'inspire librement des principes du sujet (vous gardez seulement ce qui vous convient) pour créer votre propre stratégie pour peindre. Mais ne soyez pas trop timides, juste changer la définition du poids d'une région, ou ajouter une variante en plus de A et B, ne sera pas suffisant. Ensuite vous devez implémenter votre stratégie.

2.2.5 Programme principal

Votre programme doit permettre à l'utilisateur de passer en paramètre (en ligne de commande et dans cet ordre précis) les valeurs suivantes, qui auront exactement ces noms dans votre programme :

- `strategie`, la stratégie de génération d'arbre, celle de l'énoncé (paramètre égal à 1) ou celle que vous avez créée (paramètre égal à 2). Les paramètres suivants concernent la stratégie de l'énoncé. Si votre stratégie nécessite des paramètres, il doit être possible pour l'utilisateur de les spécifier.
- `largeur`, `hauteur`, la hauteur et largeur (en nombre de pixels) de l'image générée.
- `nbFeuillesAdiviser`, le nombre de feuilles maximum à diviser.
- `minDimensionCoupe`, la taille minimum (en pixels) des dimensions d'une région pour pouvoir la diviser.
- `largeurLigne`, la largeur (en pixels) de la ligne qui sépare les régions.
- `variante`, la variante (A ou B) à utiliser pour la division.
- le nom du fichier texte en entrée.

Le programme doit s'exécuter sans aucune interaction avec l'utilisateur, et doit produire l'image correspondant aux données en entrée.
Votre code doit absolument contenir (avec ce nom) les fonctions :

- `chooseLeaf` (ou `choisirFeuille`) qui identifier la feuille à diviser

- `chooseDivision` (ou `choisirDivision`) qui identifie l'axe et la coordonnée de la division d'une feuille donnée
- `chooseColor` (ou `choisirCouleur`) qui définit la couleur d'une feuille nouvellement créée
- `generateTree` (ou `genereArbre`) qui génère un arbre selon la procédure vue plus haut
- `generateBetterTree` (ou `genereMeilleurArbre`) qui génère un arbre selon la procédure que vous avez créée
- `toImage` qui à partir d'un arbre, génère l'image au format PNG du tableau. Cette fonction doit utiliser le fichier `Image.java` qui vous est fourni.

Vous choisirez également des noms clairs pour vos structures de données, par exemple `Tree2D`, et/ou `ABR`, et/ou `AVL`, etc.

Important. Votre programme doit implémenter efficacement la procédure indiquée, en réalisant une *complexité au pire* minimum pour l'ensemble du traitement. Pour cela, à chaque étape de la procédure (et surtout dans le choix de la feuille) vous devez choisir et implémenter avec soin vos structures de données. Le code doit être commenté pour préciser vos choix et les complexités obtenues pour chaque fonction.

3 Exemples de résultats

Vous trouverez sur Madoc quelques exemples d'images générées avec la stratégie 1.

4 Rendu de Projet

Pour vous aider avec la gestion des images en Java, nous vous fournissons une classe `Image.java` avec laquelle vous pouvez créer une image, colorier des pixels ou des régions, et sauvegarder au format png. Cette classe ne doit pas être modifiée.

Les programmes doivent fonctionner parfaitement sur les machines du CIE, sous Linux. Le programme sera lancé en ligne de commande et pourra donc être utilisé indépendamment de tout IDE.

Vous déposerez avant le **samedi 27 novembre à 23h55** sur Madoc une archive (un seul dépôt par binôme) de type .zip, gzip, .gz ou .tgz qui contiendra un seul répertoire Nom1Nom2 (les deux noms de famille des membres du binôme). Ce répertoire doit contenir : un répertoire appelé `src` contenant les sources de votre programme, le fichier texte README donnant les instructions d'utilisation de votre programme (surtout pour votre propre stratégie), et des fichiers de tests (fichiers en entrée du programme et les sorties de programme correspondantes).

- La compilation doit se faire, une fois l'archive extraite dans le répertoire courant (quel qu'il soit), avec la commande:

  ```
  javac -source 1.8 -target 1.8 -d Nom1Nom2/bin Nom1Nom2/src/*.java
  ```

- L'exécution doit se faire avec la commande:

  ```
  java -classpath Nom1Nom2/bin Main [args]
  ```

  où [args] est la liste d'arguments, telle que précisée dans la Section 2.2.5. Cela signifie que le nom de votre classe contenant le programme principal (`main`) doit être Main.java.